

DESGLOSE MATEMÁTICO Y LÓGICO DEL CONTROL PREDICTIVO BASADO EN MODELO (MPC)

1. Modelo Cinemático del Robot (Diferencial)

Descripción: Es la base de todo. Define cómo cambian las coordenadas del robot en el plano XY en función de la velocidad lineal (v) y la velocidad angular (ω).

Ecuaciones (Tiempo continuo):

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} [v \omega]$$

- **Justificación:** El robot está restringido por su diseño diferencial; no puede moverse lateralmente (condición de no-holonomía). Esta matriz transforma las velocidades locales en movimiento global.

2. Discretización del Modelo

Descripción: Como el ESP32 trabaja en pasos de tiempo (según tus gráficas, $\Delta t = 0.1s$), necesitamos la versión discreta para predecir el futuro.

Cálculo (Método de Euler):

$$x_{k+1} = x_k + v_k \cos(\theta_k) \Delta t$$

$$y_{k+1} = y_k + v_k \sin(\theta_k) \Delta t$$

$$\theta_{k+1} = \theta_k + \omega_k \Delta t$$

- **Justificación:** Permite al controlador calcular la posición "estimada" en el siguiente paso para compararla con la referencia del "8".

3. Función de Costo (El "Cerebro" del MPC)

Descripción: El MPC busca las señales de control (v y ω) que minimizan esta función matemática en un horizonte de predicción N .

Ecuación de optimización:

$$J = \sum_{i=1}^N \underbrace{|x_{k+i} - x_{ref,k+i}|^2_Q}_{\text{Error de Seguimiento}} + \sum_{i=0}^{N-1} \underbrace{|u_{k+i}|^2_R}_{\text{Esfuerzo de Control}}$$

- **Justificación:**
 - **Término Q:** Obliga al robot a estar lo más cerca posible de la línea roja (referencia).
 - **Término R:** Evita que los motores den "tirones" bruscos o alcancen velocidades infinitas, suavizando la señal verde y morada de tus gráficas.

4. Matrices de Pesos (Q y R)

Descripción: Son los parámetros de sintonía que ajustaste para que tu robot no oscilara.

- **Matriz Q (Estados):** Suele ser $\text{diag}(q_x, q_y, q_\theta)$ Define qué es más importante: ¿no desviarse del camino (x, y) o mantener la orientación correcta(θ)?
- **Matriz R (Acciones):** $\text{diag}(r_v, r_\omega)$. Penaliza el uso excesivo de batería y protege la vida útil de los motores.

5. Cinemática Inversa: De Robot a Motores

Descripción: Esta es la parte que programarás en el ESP32. El MPC te da v y ω , pero tú necesitas saber cuánto voltaje mandar a la rueda derecha (V_R) y a la izquierda (V_L).

Ecuaciones de transformación:

$$v_R = v + \frac{\omega \cdot L}{2}$$
$$v_L = v - \frac{\omega \cdot L}{2}$$

Donde:

- L: Distancia entre las ruedas (ancho del robot).
- v_R, v_L : Velocidad lineal de cada rueda.
- **Justificación:** En una curva a la izquierda ($\omega > 0$), la rueda derecha debe girar más rápido que la izquierda para compensar el radio de giro.

6. Mapeo a PWM (Señal Eléctrica)

Descripción: El ESP32 usa analogWrite o ledcWrite (0 a 255). Necesitas una constante de conversión (K) que dependa de tus motores y el voltaje de tu batería.

Cálculo final:

$$PWM_R = \text{sat}(v_R \cdot K)$$

$$PWM_L = \text{sat}(v_L \cdot K)$$

- **Justificación:** La función de saturación(sat) asegura que si el cálculo pide "300", el ESP32 se mantenga en el límite máximo de "255" para no desbordar el registro.

Resumen para tu informe: El sistema utiliza un **Modelo de Estados Linealizado** alrededor de la trayectoria de referencia, permitiendo resolver un problema de **Programación Cuadrática (QP)** en cada paso de tiempo para garantizar que el error de seguimiento converja a cero